

本地活动规划与执行 Agent 设计说明

1. 项目定位

本项目面向“周末 / 短时本地活动规划”场景，目标不是生成地点推荐列表，而是构建一个能够完成“规划、确认、执行、反馈记忆”的本地生活 Agent。用户只需输入一句自然语言目标，例如“晚上想看电影再吃烧烤”“带爸妈孩子找个安静地方走走”，系统即可自动识别人群、时间、场景、饮食、距离和天气约束，生成 4-6 小时左右的可执行路线。

系统输出包括真实地点、时间轴、路线地图、预算估计、天气提示、餐厅预约、门票购买、排队查询、分享文案和异常备选方案。用户确认后，可通过 Mock API 演示“一键安排”的完整闭环；用户不满意时，也可以通过自然语言反馈让系统记住偏好，并影响下一次规划。

2. 系统架构

系统采用前后端分离架构。前端使用 Vue3 + TypeScript，提供“附近快排”和“精细规划”两种入口，并展示路线卡片、地图、预算、质检状态、异常预案和一键执行按钮。后端使用 FastAPI + HelloAgents，实现多智能体规划、工具调用、规则质检、Mock 执行和记忆检索。核心流程如下：

```
用户输入
-> 时间 / 硬需求解析
-> 场景识别 ScenarioDetector
-> Graph Memory RAG 记忆检索
-> 真实 POI / 天气 / 可用性工具调用
-> Planner 生成路线
-> 真实地点落地与规则质检
-> 前端展示与一键执行
-> 用户反馈写入记忆
```

工具层接入高德地图 POI 搜索、定位反查、天气查询，并通过 Mock API 模拟餐厅预约、门票购买、排队查询、配送和分享。记忆层使用 SQLite 保存原始反馈和结构化记忆，Neo4j 作为可选图谱展示层，形成“用户 -> 场景 -> 地点 -> 偏好标签”的画像结构。

3. Planning 策略与工具调用链

系统提供两种规划入口：

- 附近快排：**用户只输入一句话并授权当前位置，系统自动识别时间、人群、场景和偏好，优先规划 3-5 公里内或同一区域内的活动。
- 精细规划：**用户手动填写城市、区域、日期、开始时间、时长、预算和群体信息，适合约束更明确的场景。

同时，系统支持两条执行链路：

- fast 极速生成模式：**面向比赛 Demo 和即时响应场景。系统先用轻量 LLM 抽取本次硬需求，例如“电影”“烧烤”“亲子”“失恋散心”，再直接调用高德 REST POI 构建真实候选池，由轻量选择器选点，并使用确定性排程生成时间轴。该模式绕过复杂 Agent 初始化，优先保证速度和稳定性。
- agent 深度思考模式：**保留多智能体协作能力，由意图 Agent、场所搜索 Agent、餐厅搜索 Agent、天气 Agent 和 Planner Agent 协同完成复杂规划。为避免 LLM 编造地点，Planner 输出后仍会再次调用高德 POI 进行真实地点落地。

Planning 阶段的工具调用分为三类：

- 搜索与环境工具：**高德 POI 搜索、定位反查、天气查询。
- 可用性工具：**模拟餐厅是否有座、排队组数、可预约时间段。

3. **执行工具**：餐厅预约、门票购买、排队查询、配送下单、附近团购和分享消息生成。

在 agent 模式中，场所搜索、餐厅搜索、天气和可用性查询可并行执行；fast 模式则直接调用高德 REST 与 Mock 服务，减少响应耗时。

4. 质量控制与异常处理

系统在 Planner 后设置规则质检层 `RuleValidator`。原本设计希望实现 React + Reflection 式反思型 Agent，但考虑比赛时间和系统稳定性，最终将反思能力工程化为确定性的规则质检层。质检覆盖以下方面：

- **结构完整性**：是否包含城市、日期、时间轴、预算、执行动作和分享文案。
- **时间合理性**：节点是否连续，是否存在时间重叠或倒流，总时长是否接近 4-6 小时。
- **地理一致性**：地点是否与用户城市 / 区域一致，避免跨城或路线过长。
- **活动组成**：是否包含玩乐、餐饮和交通，而不是单点推荐。
- **执行闭环**：餐饮节点是否有预约动作，购票节点是否有购票动作。
- **场景匹配**：亲子场景避开酒吧、恐怖密室；失恋散心避开强社交和庆祝类安排。
- **真实 POI**：非交通节点必须绑定真实高德 POI，包括名称、地址、坐标和可选 poi_id。

fast 模式只允许从真实高德候选池中选点。如果真实 POI 不足，系统会返回候选不足的质检失败态，而不会生成虚构地点。

异常处理方面，系统在前端显式展示“异常预案”，并在执行阶段提供订单式状态流转：

待确认 -> 执行中 -> 成功 / 失败 -> 重试 / 改用备选

当前覆盖的异常包括：

- **餐厅无座**：返回附近同类型或同场景备选餐厅。
- **门票无票**：替换为免费、免预约或同类型活动。
- **时间冲突 / 超时**：压缩停留时间、删除可选节点或替换更近地点。
- **天气不适合**：下雨、高温或强风时优先替换为商场、影院、展览、书店咖啡馆等室内活动。

预约、购票、排队、配送和团购均通过 Mock API 模拟，不会产生真实订单、支付或商户请求，但可以完整展示“规划 -> 确认 -> 执行 -> 失败 -> 备选重试”的本地生活闭环。

5. Graph Memory RAG 记忆系统

系统没有直接保存完整聊天作为长期偏好，而是将用户输入、反馈和执行行为凝练为三层结构化记忆：

- **用户级记忆**：跨场景稳定偏好，例如少排队、不太远、轻松不折腾。
- **场景级记忆**：只在当前场景下使用，例如约会喜欢出片、亲子喜欢儿童友好、朋友喝酒偏好夜宵。
- **对象级记忆**：针对具体地点或餐厅的喜欢 / 避雷，例如某餐厅太贵、某公园适合孩子。

规划前，`ScenarioDetector` 先识别当前场景，例如 `couple_date`、`family_kid`、`friend_drink`、`solo_healing`。随后 `MemoryService` 按当前用户和场景检索相关记忆，形成 Graph Memory RAG 上下文注入 Planner。这样系统可以记住“这个用户不喜欢太远”，但不会把“亲子餐厅”错误迁移到“兄弟喝酒”场景中。

Neo4j 图谱结构为：

User -> UserScenario -> Place -> Preference

SQLite 负责可靠保存原始反馈事件和结构化记忆，Neo4j 负责展示凝练后的用户画像图。系统还会过滤“真实 POI”“交通”等内部技术标签，避免它们被误认为用户偏好。